

Simple RAG - High Level Design Document

1. Overview

The Simple RAG (Retrieval-Augmented Generation) system is an end-to-end application designed to ingest various types of documents, generate vector embeddings for their contents, store them efficiently, and perform context-aware question answering based on user queries.

The system has both a Command Line Interface and a Flask-based Web Interface to interact with the capabilities.

2. High-Level Architecture Pipeline

The workflow of the application is typically structured into two main phases: Indexing and Retrieval/Generation.

Phase 1: Indexing Phase

1. Document ingestion: System parses files in raw data directory.
2. Content Splitting: Long texts are partitioned into smaller chunks.
3. Embedding: The chunks are processed by a neural model to produce dense vector representations.
4. Indexing: The vectors and metadata are stored in a FAISS vector database.

Phase 2: Retrieval and Generation Phase

1. User Query: User specifies a question via Web UI or CLI.
2. Query Vectorization: The system converts the query into a dense vector using the same embedding model.
3. Semantic Search: FAISS retrieves the top K most similar text chunks based on vector distance.
4. Prompt Construction: A prompt is formed containing the retrieved context and the user's query.
5. LLM Synthesis: The LLM processes the prompt and returns a coherent summarized answer.

3. Detailed Component Breakdown

3.1 Data Loader (src/data_loader.py)

Responsible for reading raw files from the 'data' directory and converting them into structured LangChain 'Document' instances. It supports dynamically loading PDFs (PyPDFLoader), TXTs (TextLoader), CSVs (CSVLoader), Excel Spreadsheets (UnstructuredExcelLoader), Word Docs (Docx2txtLoader), and JSONs. This acts as the main entry point to populate the knowledge base.

Simple RAG - High Level Design Document

3.2 Embedding Pipeline (src/embedding.py)

Given the loaded LangChain Documents, this class chunks the text using 'RecursiveCharacterTextSplitter' (e.g., chunks of 1000 characters with an overlap of 200). It then leverages the 'SentenceTransformer' library using the 'all-MiniLM-L6-v2' model to map each chunk of text into a high-dimensional vector space.

3.3 Vector Store (src/vectorstore.py)

A wrapper around the FAISS (Facebook AI Similarity Search) library. It takes the text embeddings and corresponding metadata and saves them persistently to the disk (in the 'faiss_store' folder). It supports rapid nearest-neighbor (L2 distance) searches when queried with a new embedding vector, returning the most pertinent original text chunks.

3.4 RAG Search Engine (src/search.py)

This is the integration component for retrieval and generation. When initialized, it ensures the FAISS store is ready. On a search request, it queries the vector database for the top matches. It then bundles these text matches into an LLM prompt and sends them to the Groq API (running 'llama-3.1-8b-instant') via LangChain's 'ChatGroq', yielding a synthesized, conversational answer.

3.5 Applications APIs (app.py & main.py)

- main.py: Serves as the interactive CLI application where developers can test indexing, search logic, and directly input queries in the terminal.
- app.py: A Flask web microservice containing a root frontend route ('/') and a REST API endpoint ('/search') accepting POST requests with JSON payloads. This serves the final user experience.

4. Technology Stack

- Framework: Flask (Web), LangChain (Orchestrating RAG)
- LLM Provider: Groq API (Llama3.1 models)
- Vector Database: FAISS (local index)
- Embedding Models: SentenceTransformers (all-MiniLM-L6-v2)
- Document Loaders: PyPDF, docx2txt, unstructured